



稀疏编码

主讲人：吴昊

经典实验

意大利罗马大学的Fabio Babiloni教授曾经做过一项实验，他们让新飞行员驾驶一架飞机并采集了他们驾驶状态下的脑电，同时又让老飞行员驾驶飞机并也采集了他们驾驶状态下的脑电。

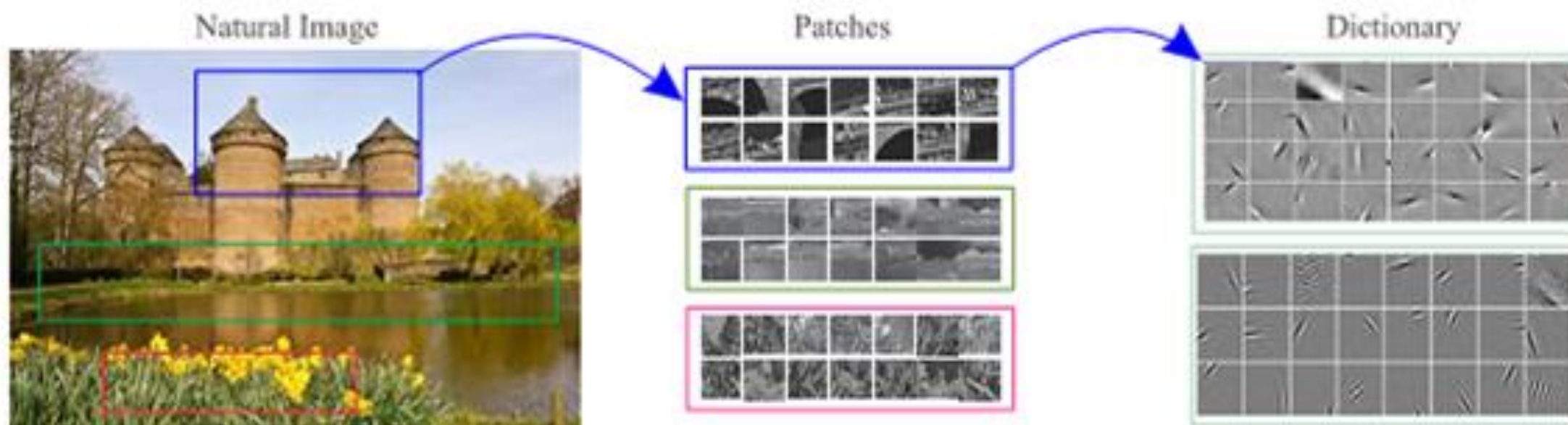
结论是熟练者的大脑（右图）可以调动尽可能少的脑区,消耗尽可能少的能量,进行同样有效的计算。并且由于调动的脑区很少，大脑计算速度也会变快，这就是我们称熟练者为熟练者的原理所在。站在我们所要理解的稀疏字典学习的角度上来讲，就是大脑学会了知识的稀疏表示。



稀疏表示的本质：用尽可能少的资源表示尽可能多的知识，这种表示还能带来一个附加的好处，即计算速度快。

稀疏编码简介

稀疏编码通俗的说，就是将一个信号表示为一组基的线性组合，而且要求只需要较少的几个基就可以将信号表示出来。“稀疏性”定义为：只有很少的几个非零元素或只有很少的几个远大于零的元素。比如自然图像，可以被表示成少量基本元素的叠加。



字典学习

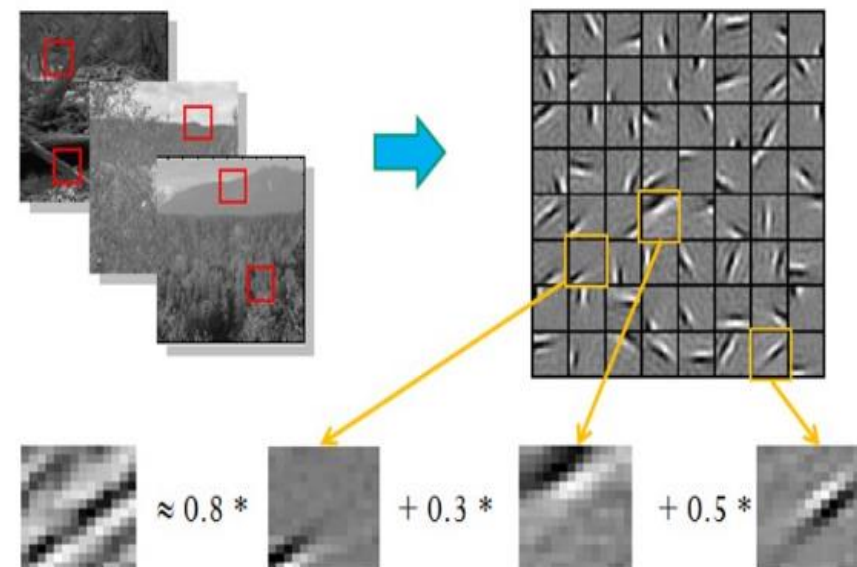
我们都知道稀疏编码可以分为两大步骤：（1）字典训练，即利用训练样本通过最小化代价函数得到字典（其中的“单词”也称为基等），也可称为特征学习；（2）特征编码，即利用步骤（1）中学习到的字典再次最小化代价函数，得到新的样本的表征方式。

训练阶段

$$\min_{a, \phi} \sum_{i=1}^m \left\| x_i - \sum_{j=1}^k a_{i,j} \phi_j \right\|^2 + \lambda \sum_{i=1}^m \sum_{j=1}^k |a_{i,j}|$$

编码阶段

$$\min_a \sum_{i=1}^m \left\| x_i - \sum_{j=1}^k a_{i,j} \phi_j \right\|^2 + \lambda \sum_{i=1}^m \sum_{j=1}^k |a_{i,j}|$$



字典学习

字典学习 (Dictionary Learning) 和稀疏表示 (Sparse Representation) 在学术界的正式称谓应该是稀疏字典学习 (Sparse Dictionary Learning)。该算法理论包含两个阶段：字典构建阶段 (Dictionary Generate) 和利用字典 (稀疏的) 表示样本阶段 (Sparse coding with a precomputed dictionary)

问题1：我们为什么需要字典学习？

知识爆炸式增长、科技不断进步，一本新华字典或牛津字典足以表达人类从古至今乃至未来的所有知识，那些知识只不过是字典中字的排列组合。

字典学习的好处

- 它实质上是对于庞大数据集的一种降维表示，或者说是信息的压缩
- 正如字是句子最根本、典型的特征一样，字典学习总是尝试学习蕴藏在样本背后最根本、典型的特征。

上述原因解释了为何在卷积神经网络关注度极高的情况下，稀疏编码仍然有许多学者、工程人员持续关注和研究。

参考<https://blog.csdn.net/fuzhenqi/article/details/77966857>

稀疏表示

问题2：我们为什么需要稀疏表示？

回答这个问题毫无疑问就是要回答“稀疏字典学习”中稀疏两字的来历。相信大部分人都有这样一种感觉，当我们在解涉及到新的知识点的数学题时总有一种累心（累脑）的感觉。但是当我们通过艰苦卓绝的训练将新的知识点牢牢掌握时，再解决与这个知识点相关的问题时就不觉得很累了。这是为什么呢？

正如意大利罗马大学的Fabio Babiloni教授的实验中所述，站在我们所要理解的稀疏字典学习的角度上来讲就是大脑学会了知识的稀疏表示。稀疏表示的本质：用尽可能少的资源表示尽可能多的知识，这种表示还能带来一个附加的好处，即计算速度快。

其实，我们不仅希望用字典中的单词完备地表征知识。而且，希望一段知识被表征所用到的单词越少（精炼）越好。

参考<https://blog.csdn.net/fuzhenqi/article/details/77966857>

稀疏优势

1) 特征选择(Feature Selection)

稀疏能实现特征的自动选择。一般来说， x_i 的大部分元素（也就是特征）都是和最终的输出 y_i 没有关系或者不提供任何信息的。在最小化目标函数的时候考虑 x_i 这些额外的特征，虽然可以获得更小的训练误差，但在预测新的样本时，这些没用的信息反而会被考虑，从而干扰了对正确 y_i 的预测。稀疏规则化算子的引入会完成特征的自动选择，它会学习地去掉这些没有信息的特征，也就是把这些特征对应的权重置为0。

2) 可解释性(Interpretability)

稀疏能够增加模型的可解释性。例如患某种病的概率是 y ，然后我们收集到的数据 x 是1000维的，也就是我们需要寻找这1000种因素到底是怎么影响患上这种病的概率的。假设我们这个是个回归模型： $y = w_1 * x_1 + w_2 * x_2 + \dots + w_{1000} * x_{1000} + b$ （当然了，为了让 y 限定在 $[0,1]$ 的范围，一般还得加个Logistic函数）。通过学习，如果最后学习到的 w_i 就只有很少的非零元素，例如只有5个非零的 w_i ，那这些对应的特征对是否患病起到决定性的作用。换句话说，是否患病主要取决于这5个因素。

参考<https://www.zhihu.com/question/49426178>

稀疏编码思想的流程

稀疏编码的核心内容是寻找一组完备的基向量，该基向量能够准确地对不同的图像信息进行编码和表达，如公式（1）所示：

$$I = \sum_{i=1}^k a_i v_i \quad (1)$$

为待编码图像，编码的目的是利用一组基向量 $[v_1, v_2, \dots, v_k]$ 和稀疏系数 $[a_1, a_2, \dots, a_k]$ 对待编码图像进行近似精确的表达。

稀疏编码思想的流程

首先从图像库中选取有代表性、复杂程度较高的图像用来对基向量进行训练。假设给定的样本图片为 $[x_1, x_2 \dots x_k]$ ，我们需要学习一组基向量 $[v_1, v_2 \dots v_k]$ ，也就是经常说到的字典。基向量的获取过程，是稀疏编码的一个核心部分，训练过程往往直接决定着基向量的获取质量。训练过程是一个重复迭代的过程，在训练过程中需要寻找最佳基向量和参数使得目标函数 E_1 最小，如公式 (2) 所示：

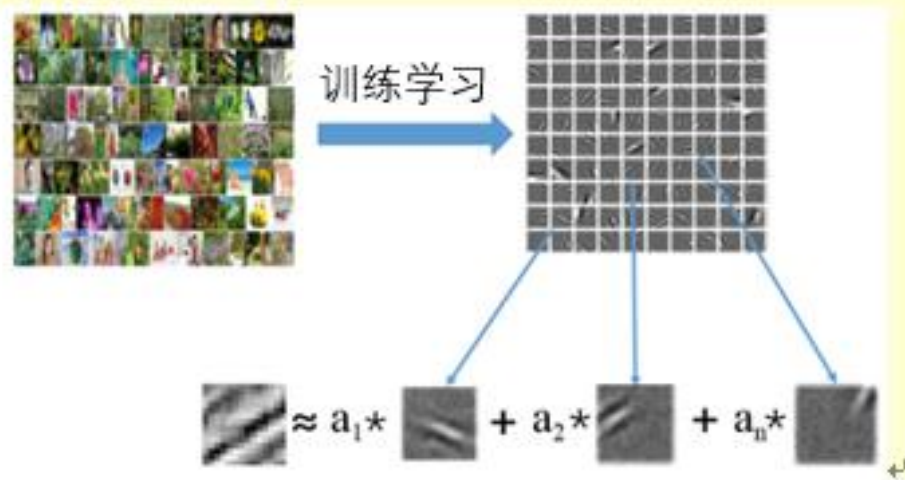
$$E_1 = \min_{a,v} \sum_{i=1}^m |x_1 - \sum_{j=1}^k a_{i,j} v_j|^2 + \lambda \sum_{i=1}^m \sum_{j=1}^k |a_{i,j}| \quad (2)$$

在公式 (2) 中， x_i 是输入图像， $a_{i,j}$ 为稀疏编码的系数， v_j 为字典中的基向量。我们交替地改变 $a_{i,j}$ 和 v_j ，并依次地调整 i, m, j, k 等参数，使得目标函数最小。迭代过程具体可以分为两步：固定字典中的基向量 v_j ，调整编码系数 $a_{i,j}$ ，使得目标函数最小；固定编码系数 $a_{i,j}$ ，调整字典中的基向量 v_j ，使得目标函数最小。在不断的迭代过程中，直至收敛，可以得到能够良好表达样本图像的一组基向量。在得到了一组基向量之后，接下来我们利用基向量对图像进行充分表达，也就是编码。在编码过程中，需要利用上述基向量对待表达图像进行编码，具体过程如公式 (3) 所示：

$$E_2 = \min_a \sum_{i=1}^m |x_1 - \sum_{j=1}^k a_{i,j} v_j|^2 + \lambda \sum_{i=1}^m \sum_{j=1}^k |a_{i,j}| \quad (3)$$

稀疏编码思想的流程

相比于公式 (2) , 公式 (3) 中并不需要对基向量进行求解, 只需要对其它参数进行获取, 从而得到稀疏向量。虽然, 基向量已经被锁定, 但其它参数 i,m,j,k 仍然需要逐步调整, 在逐步调整的过程中实现目标函数的最小化。当目标函数最小时, 我们对此时的编码系数 $a_{i,j}$ 进行获取, 从而得到该图像的稀疏编码表达。如下图所示, 该示意图展示了基向量的获取过程和图像的特征深度提取过程。



图注：图像特征深度提取过程。该过程通过对海量图像进行训练学习从而得到一组基向量，该组基向量可以将图像进行近似精确表达。

其它常见稀疏性结构

关于常见的一些结构性稀疏，列举如下：

Hierarchical Sparse Coding[[code](#) | [read more](#)]，认为非0的系数之间存在层次结构，即group与group之间要么不overlap，如果overlap则一个group必会包含另一个group。一种典型的层次结构就是tree结构

Overlapping group sparse coding[[code](#) | [read more](#)]，则将约束relax，即允许group之间相互overlap。这个模型据说在genetic data的描述中十分有效，大家不妨试试

Graph-Guided Sparse Coding[[code](#) | [read more](#)]，建立一个graph，graph中各结点是dictionary中的各个atom。Graph-Guided不同于以上的group sparse coding，它可以加入更加复杂的结构信息。